MQTT vs LwM2M Playground

An embedded-C IoT protocol laboratory on a PC
Design, architecture, workflow and measurement approach

Phase 1 – simulated devices, real servers

October 2026

Abstract

This document describes a software laboratory for learning and benchmarking two IoT device-management and telemetry protocols, MQTT and LwM2M, before any hardware is available. Simulated sensor devices are written in C with the same libraries an STM32 would use (Eclipse Paho MQTTPacket, Eclipse Wakaama, mbedTLS) and talk to real servers (Eclipse Mosquitto and Eclipse Leshan). A small C web server (the *lab*) starts the devices, observes both servers and streams every protocol message to an interactive browser dashboard. The document covers the goals and constraints, the architecture, every component, the end-to-end workflow, how measurements are taken, the network and security experiments, the simulated OTA exercise, how the system was verified, the problems found during development and how they were fixed, measured results, known limitations, and the plan for porting to STM32 + Quectel EC200U.

Contents

1 Goals and constraints **3**
- 1.1 What the laboratory is for . . . 3
- 1.2 Constraints that shaped the design . . . 3

2 Overall architecture **3**
- 2.1 Why this split . . . 4

3 Device software architecture **4**
- 3.1 Platform abstraction layer (`pal.c`) . . . 4
- 3.2 Transport (`link.c`) . . . 4
- 3.3 Simulated sensor (`sensor.c`) . . . 5
- 3.4 Telemetry payload (`telemetry.c`) . . . 5
- 3.5 Event reporting and control (`events.c`, `control.c`) . . . 5

4 MQTT device **5**
- 4.1 Approach . . . 5
- 4.2 Session flow . . . 5
- 4.3 QoS flows and latency . . . 6

5 LwM2M device **6**
- 5.1 Approach . . . 6
- 5.2 Objects . . . 6
- 5.3 Transactions . . . 7
- 5.4 CoAP tracing . . . 7
- 5.5 Confirmable notifications . . . 7

6 The lab server and dashboard **7**
6.1 Lab server (C) . . . 7
6.2 Dashboard . . . 8

7 End-to-end workflow **9**
7.1 Build . . . 9
7.2 Start the servers . . . 9
7.3 Run . . . 9
7.4 Data flow of one MQTT message . . . 9

8 Measurement methodology **10**
8.1 What is recorded . . . 10
8.2 Latency . . . 10
8.3 Statistics . . . 10
8.4 Connection cost . . . 10

9 Network simulation **10**
9.1 Built-in model (all platforms) . . . 10
9.2 Kernel model (Linux/WSL) . . . 11

10 Security experiments **11**

11 Simulated OTA **11**

12 Verification approach **11**

13 Problems found and how they were solved **12**

14 Measured results (development PC) **12**

15 Known limitations **13**

16 Porting to STM32 + EC200U **13**

A Repository layout **14**

B Command-line reference **14**

C Leshan REST calls used **15**

1 Goals and constraints

1.1 What the laboratory is for

- Learn MQTT and LwM2M *by interacting with them*: see every message (`CONNECT`, `PUBLISH`, `REGISTER`, `NOTIFY`, ...), its bytes and its timing.
- Compare the two protocols with honest measurements: latency, bytes on the wire, connection cost, success rate.
- Experiment with bad networks (latency, packet loss) and with security (TLS, DTLS) and see the effect immediately.
- Produce code that can later be moved to an STM32 with an EC200U cellular modem without rewriting the protocol logic.

1.2 Constraints that shaped the design

- **No hardware.** Everything runs on a Windows PC; Linux/macOS/WSL must work too.
- **Devices in C, no Python.** The simulated devices must be embedded-style C so the code is portable to a microcontroller. Python must not be required to run anything.
- **Real servers.** Mosquitto and Leshan, not mocks.
- **No fabricated numbers.** A value that was not measured is shown as "Not measured"; a failed message has no latency.
- **Small and playable first.** Milestones in order: MQTT sample, LwM2M sample, dashboard, benchmark, security, network simulation, OTA.

2 Overall architecture

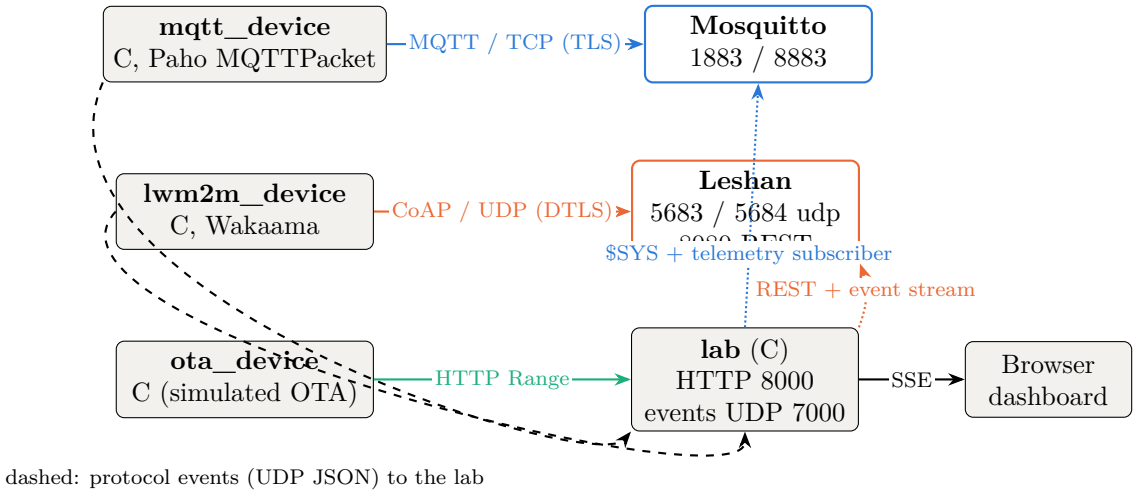


Figure 1: Processes and connections. Each device is a separate program; the lab starts them, receives their events, watches both servers and serves the dashboard.

Four kinds of programs run on the PC:

1. **Simulated devices** (`mqtt_device`, `lwm2m_device`, `ota_device`) – C programs, one per protocol. Each can run on its own from a terminal or be started by the lab.
2. **Real servers** – Mosquitto (MQTT broker) and Leshan (LwM2M server), started by scripts or Docker.

3. **The lab** – a C web server that hosts the dashboard, relays device events to the browser, starts/stops devices, proxies Leshan's REST API, keeps its own view of both servers, serves OTA images and stores benchmark results.
4. **The browser dashboard** – HTML, CSS and plain JavaScript with no external libraries (works offline); visualisation only.

2.1 Why this split

- Devices stay small and portable: no web or UI code inside them, only a one-way event side channel that can be turned off (`-lab off`).
- Devices can be used without the dashboard, exactly like firmware on a bench with a serial console.
- The server-side truth is observed independently of the device: the lab subscribes to the broker and follows Leshan's event stream, so the dashboard can show what the *server* received next to what the *device* believes it sent.

3 Device software architecture

Application `mqtt/main.c`, `lwm2m/main.c`, `ota/ota_device.c` – sampling loop, commands, run control

Protocol `mqtt_client.c` (Paho MQTTPacket) `lwm2m_client.c` + `objects.c` (Wakaama) `coap_trace.c`

Services `sensor.c`, `telemetry.c`, `bench.c`, `events.c`, `control.c`, `cli.c`

Transport `link.c`: optional TLS (TCP) / DTLS (UDP) via mbedTLS, byte and handshake accounting

Network simulation `netsim.c`: latency/loss delay lines (only active when configured)

PAL `pal.c`: time, sockets, PRNG – *the only OS-specific file; replaced on STM32 + EC200U*

Figure 2: Layers inside every simulated device. Each layer only calls the one below.

3.1 Platform abstraction layer (`pal.c`)

All operating-system access lives here: monotonic and wall-clock time, sleep, sockets (connect, send, receive with timeout, close, local UDP bind, TCP listen/accept) and a deterministic xorshift PRNG. Windows (Winsock) and POSIX are both implemented. The socket API is deliberately the same shape as a cellular modem's AT socket service, which is what makes the later port cheap (Section 16).

3.2 Transport (`link.c`)

A `link_t` is a connection to one server:

- TCP or UDP; with security enabled, TLS over TCP or DTLS over UDP using mbedTLS 3.6.

- Counts **wire bytes** (what crosses the socket, including TLS/DTLS record overhead), packets, TCP connect time, handshake time and handshake bytes in each direction.
- TLS: verifies the broker certificate against a CA file and expected name; TLS 1.3 by default, `-tls12` caps at 1.2.
- DTLS: pre-shared key, cipher suites `TLS_PSK_WITH_AES_128_CCM_8` and `TLS_PSK_WITH_AES_128_CBC_SHA2` RFC 6347 retransmission timers.

The protocol code only calls `link_send` / `link_recv`; it does not know whether the link is encrypted or impaired.

3.3 Simulated sensor (`sensor.c`)

`SensorData_t` holds temperature (20–35 °C), humidity (30–90 %), pressure (990–1030 hPa) and battery (0–100 %). Each channel is a mean-reverting random walk (discrete Ornstein–Uhlenbeck process) clamped to its range, so values drift gradually. Humidity is weakly anti-correlated with temperature; the battery drains slowly and is "swapped" at 0 %. A seed (`-seed`) makes the sequence fully reproducible.

3.4 Telemetry payload (`telemetry.c`)

Both protocols carry the same JSON document:

```
{"dev":"mqtt-dev-1","seq":12,"ts":1730000000000,"temperature":27.41,
 "humidity":62.10,"pressure":1012.30,"battery":87}
```

For payload-size experiments a `"pad"` field is added so the document is exactly the requested number of bytes.

3.5 Event reporting and control (`events.c`, `control.c`)

Every protocol message is reported twice: as a human-readable console line and as a JSON datagram to the lab (UDP 127.0.0.1:7000). A JSON event contains the protocol, direction, message type, lengths, QoS, latency, security label, a short detail text, the payload (truncated) and the raw bytes as hex. Devices also listen on a local UDP control port (7101 MQTT, 7102 LwM2M, 7103 OTA) for text commands such as `stop`, `notify`, `deregister`, `interrupt`. Both channels are lab plumbing, not part of either protocol.

4 MQTT device

4.1 Approach

Eclipse Paho's **MQTTPacket** library only serialises and parses MQTT packets; it does no I/O. The playground owns the socket (`mqtt_client.c`), so it can time each exchange precisely and count every byte. This is also the usual structure on a microcontroller.

4.2 Session flow

```
DEVICE                                  BROKER
  | TCP connect                            |
  | [TLS handshake]          (--secure)    |
  | CONNECT client_id keepalive clean ----> |
  | <---------------------- CONNACK rc=0    |
  | SUBSCRIBE playground/<id>/cmd QoS1 ---> |
  | <--------------------------- SUBACK     |
  | PUBLISH playground/<id>/telemetry ----> |  every --interval
```

```
|      ... acknowledgement per QoS ...    |
| PINGREQ / PINGRESP  (keep-alive)        |
| DISCONNECT ---------------------------> |
```

QoS 0, 1 and 2 are supported in both directions. The command topic lets the broker deliver commands (`interval=`, `qos=`, `publish`, `pause`, `resume`, `stop`) to the device, which demonstrates the subscribe path.

4.3 QoS flows and latency

QoS	Packets	Latency definition
0	PUBLISH	PUBLISH → PINGRESP of a trailing PINGREQ
1	PUBLISH, PUBACK	PUBLISH → PUBACK
2	PUBLISH, PUBREC, PUBREL, PUBCOMP	PUBLISH → PUBCOMP

QoS 0 has no acknowledgement, so "broker receipt" is not directly observable. MQTT runs over a single ordered TCP stream that the broker reads in order; a PINGREQ sent immediately after the PUBLISH can only be answered after the PUBLISH has been read. Its PINGRESP therefore proves receipt. The probe costs 4 bytes per message, which appear in the byte counts; `-no-probe` disables it and the latency is then reported as not measured.

5 LwM2M device

5.1 Approach

Eclipse **Wakaama** (pinned to the 2026-05-26 snapshot, commit `94ff56f`) implements the LwM2M state machine, CoAP and the data formats (TLV, SenML JSON/CBOR). The playground supplies Wakaama's platform and transport callbacks (`lwm2m_connect_server`, `lwm2m_buffer_send`, `lwm2m_close_connection`, memory and time functions) on top of `link_t`, and implements the objects.

5.2 Objects

Path	Meaning	Ops	Notes
`/0/0`	Security: server URI, mode (PSK/NoSec), keys	–	never readable by a server
`/1/0/1`	Server: lifetime	RW	default 300 s
`/1/0/8`	Server: registration update trigger	E	sends UPDATE
`/3/0/0..3`	Device: manufacturer, model, serial, FW version	R	
`/3/0/4`	Device: reboot	E	simulated
`/3/0/9`	Device: **battery level** (%)	R	battery lives here
`/3/0/11`	Device: error code	R	multi-instance
`/3/0/13..15`	Device: current time, UTC offset, timezone	RW	
`/19/0/0`	BinaryAppDataContainer: data	R	telemetry JSON, sized by `-payload`
`/3303/0/5700`	Temperature value (Cel)	R	also min/max, range, units
`/3303/0/5605`	Temperature: reset min/max	E	
`/3303/0/5750`	Temperature: application type	RW	good Write target

Path	Meaning	Ops	Notes
/3304/0/5700	Humidity (%RH)	R	same resource set
/3315/0/5700	Barometer (hPa)	R	same resource set

OMA's core object set has no stand-alone battery object; battery level is resource 9 of the Device object (/3/0/9). /3/0/13 is Current Time.

All objects share one **table-driven engine** (objects.c): an object is a resource table (id and allowed operations) plus get/set/execute callbacks; read, write, execute and discover are implemented once. This keeps the code small, which also suits a microcontroller.

5.3 Transactions

```
REGISTER    D -- CON POST /rd?ep=..&lt=300&lwm2m=1.1&b=U  (link-format) --> S
            D <-------------------------- ACK 2.01 Created, Location /rd/<id>
READ        S -- CON GET  /3/0/9 ----------> D -- ACK 2.05 + value -------> S
WRITE       S -- CON PUT  /3303/0/5750 ----> D -- ACK 2.04 ----------------> S
EXECUTE     S -- CON POST /3303/0/5605 ----> D -- ACK 2.04 ----------------> S
OBSERVE     S -- CON GET  /3303/0/5700 Obs:0 -> D -- ACK 2.05 Obs:n + value -> S
NOTIFY      D -- CON 2.05 Obs:n+1 + value (same token) ------------------> S
            D <------------------------------------------- empty ACK -- S
DEREGISTER  D -- CON DELETE /rd/<id> ---------------------------------> S
            D <---------------------------------------------- ACK 2.02 -- S
```

5.4 CoAP tracing

coap_trace.c decodes every datagram Wakaama sends or receives (type, code, Message ID, token, Uri-Path/Query, Observe, Content-Format) and names it the way the LwM2M specification does (REGISTER, OBSERVE, NOTIFY, READ RESPONSE, ...). A small table of outstanding confirmable requests per direction matches each ACK to its request by Message ID, which yields the latency and detects retransmissions (same Message ID sent again).

5.5 Confirmable notifications

Wakaama sends Observe notifications as **NON** (non-confirmable). Nothing acknowledges them, so neither delivery nor latency can be measured and a lost notification is silently gone. RFC 7641 allows either type, so the playground flips each outgoing notification to **CON** and takes over its reliability using RFC 7252 timing:

- initial timeout ACK_TIMEOUT $\times$ random$[1, 1.5]$ (2–3 s by default);
- retransmit the same message (same Message ID), doubling the timeout;
- give up after MAX_RETRANSMIT $= 4$ retransmissions (62–93 s after the first transmission).

Latency is CON $\rightarrow$ ACK measured from the *first* transmission. -notify-non restores the stock behaviour, in which case latency is honestly reported as not measured.

A notification can end in RST, which counts as a failure: Leshan rejects a retransmitted notification once it already has a newer one, and notifications in flight are rejected when a second OBSERVE replaces the observation (new token).

6 The lab server and dashboard

6.1 Lab server (C)

lab is a small multi-threaded HTTP/1.1 server bound to 127.0.0.1:

Route	Purpose
`/`, static files	dashboard (`dashboard/www`)
`/api/events`	Server-Sent Events: every device/lab/server event, with a 3000-event replay buffer for new browser tabs
`/api/state`	snapshot of broker, Leshan and device state
`/api/device/start|stop|cmd`	start a device with validated parameters, stop it gracefully (DISCONNECT / De-register) or send a control command
`/api/mqtt/publish`	publish via the lab's broker connection (device commands)
`/api/runs`, `/results/*.csv`	benchmark summaries and per-message CSV files
`/leshan/...`	reverse proxy to Leshan's REST API and event stream
`/ota/manifest`, `/ota/<image>`	SIMULATED OTA images with HTTP Range support
`/docs/*`	the Markdown documentation

Background threads:

- **Event receiver**: UDP 7000; stores the latest state per device, appends run summaries to `results/runs.jsonl`, broadcasts everything to the browsers.
- **MQTT monitor**: its own MQTT connection; subscribes to `$SYS/broker/...` statistics and `playground/+/telemetry`, so the dashboard shows what the broker actually delivered.
- **Leshan monitor**: polls `/api/clients` and follows Leshan's `/api/event` stream (registrations, notifications) using HTTP/1.0 so responses are never chunked.
- **Reaper**: notices when device processes exit.

Device parameters coming from the browser are whitelisted per device and restricted to simple tokens before a process is spawned (no shell is involved). When an LwM2M device is started with security on, the lab first registers its PSK in Leshan; with security off it removes any stale PSK.

6.2 Dashboard

Plain HTML/CSS/JavaScript, no external libraries. Tabs:

- **MQTT**: QoS 0/1/2 selector, interval, count, payload size, security toggle, network profile; START/STOP/RESET; animated sequence diagram (DEVICE ↔ BROKER); the selected QoS's message flow; broker panel (from $SYS and the lab subscriber); device panel; live latency chart.
- **LwM2M**: same controls plus CON/NON; device operations (Register, Deregister, Notify) and server operations through Leshan (Read, Write, Observe, Cancel observe, Execute) on a selectable resource table; sequence diagram (DEVICE ↔ SERVER); Leshan panel; auto-OBSERVE after registration.
- **MQTT vs LwM2M**: side-by-side table of the latest run of each ("Not measured" when absent), one-click experiments, six charts and a table of all runs with CSV links.
- **OTA**: image and chunk size, network profile, START / INTERRUPT / RESET, animated step list, progress bar and statistics, size-vs-time chart.
- **Event log and message inspector** (all tabs): click any event to see protocol, direction, type, lengths, security, timestamp, latency, CoAP header fields and a hex dump of the raw bytes.

Charts are hand-written SVG with a colour-vision-checked palette (MQTT blue, LwM2M orange, OTA aqua), a legend and direct labels, and hover tooltips. Redraws are coalesced (at most about

eight per second) so bursts of events stay cheap, and the experiment runner advances on the lab's "device exited" event, which is not slowed by browsers' background-tab timer throttling.

7 End-to-end workflow

7.1 Build

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

The first configure fetches mbedTLS 3.6.7, Paho Embedded-C 1.1.0 and Wakaama (pinned commit) with CMake FetchContent. Outputs: `mqtt_device`, `lwm2m_device`, `ota_device`, `lab`, unit tests, and the sample images `build/ota/firmware_v1.bin` (64 KiB) and `firmware_v2.bin` (256 KiB).

7.2 Start the servers

Option	Commands
Docker	`docker compose up` (official `eclipse-mosquitto:2` image; Leshan built from `servers/leshan/Dockerfile`)
Windows, no Docker	`scripts\get-servers.ps1` once (portable Mosquitto, Java, Leshan in `servers\.local`), then `start-mosquitto.ps1` and `start-leshan.ps1`
Linux/macOS/WSL	`scripts/get-servers.sh` once, then `start-mosquitto.sh` and `start-leshan.sh`

On the development PC, Windows (Hyper-V/WinNAT) reserves TCP ports 1827–1926, which includes 1883. Either reserve 1883 with `netsh int ipv4 add excludedportrange` (admin) or use other ports, e.g. `start-mosquitto.ps1 -Port 21883 -TlsPort 28883` and `lab -mqtt-port 21883 -mqtts-port 28883`.

7.3 Run

1. Start `lab` and open `http://127.0.0.1:8000`; both server indicators must read ONLINE.
2. MQTT tab → START: watch CONNECT → CONNACK → SUBSCRIBE → SUBACK → PUBLISH → PUBACK.
3. LwM2M tab → START: REGISTER → 2.01, automatic OBSERVE, then NOTIFY → ACK every interval; try Read, Write, Execute.
4. MQTT vs LwM2M → "Benchmark: 100 + 100", then the loss, latency, payload and security experiments.
5. OTA tab → START OTA, then INTERRUPT on a poor network.

7.4 Data flow of one MQTT message

1. The device samples the sensor, builds the JSON payload, serialises a PUBLISH with Paho, timestamps it and hands it to `link_send`.
2. `netsim` (if active) holds the packet; TLS encrypts it; the PAL writes it to the socket. Wire bytes are counted.
3. Mosquitto delivers it to the lab's subscriber and answers PUBACK.
4. The device reads the PUBACK, computes PUBLISH → PUBACK latency and records one benchmark row.

5. For both packets the device emits a console line and a JSON event to the lab, which pushes them to the browser; the broker panel updates from the lab's subscriber and $SYS counters.

8 Measurement methodology

8.1 What is recorded

Per message (`results/<run>.csv`): `seq, timestamp_ms, latency_ms, payload_bytes, bytes_sent, bytes_received, success, retries`. Bytes are *wire* bytes of that message's whole exchange (including TLS/DTLS overhead, PUBREL, the QoS 0 probe and CoAP retransmissions). Per run (`results/runs.jsonl`): configuration, counts, success rate, min/avg/median/P95/P99/max latency, average bytes, connection time, handshake time and bytes.

8.2 Latency

Protocol	From → to
MQTT QoS 0	PUBLISH → PINGRESP of the trailing PINGREQ
MQTT QoS 1	PUBLISH → PUBACK
MQTT QoS 2	PUBLISH → PUBCOMP
LwM2M	CON → ACK of each notification, from its first transmission

All timestamps are taken *before* the message is handed to the network layer, so simulated delay is always inside the measured interval.

8.3 Statistics

Statistics are computed over successful messages only. Percentiles use the **nearest-rank** method: the p-th percentile of n sorted values is the $\lceil p\,n/100 \rceil$-th value. With 100 messages P99 is the 99th value; with 20 messages P95 and P99 both equal the maximum, so small runs say little about the tail. A failed message counts against the success rate and never gets a latency. A value that was not measured is serialised as `null` and shown as "Not measured".

8.4 Connection cost

Time to ready = socket setup + TLS/DTLS handshake + protocol session (CONNECT/CONNACK for MQTT, REGISTER/2.01 for LwM2M). Handshake time and handshake bytes are reported separately.

9 Network simulation

9.1 Built-in model (all platforms)

`netsim` sits between the protocol and the socket, below TLS/DTLS, and implements a **delay line in each direction**:

- **Latency**: every outgoing packet waits for the one-way latency before reaching the socket; every incoming packet waits for it before the device sees it. A request/response therefore costs $2L$ plus the real round trip, a QoS 2 exchange $4L$, a DTLS 1.2 PSK handshake (three round trips) $6L$, independent of how many packets are in flight; a server-initiated request arrives L late.
- **Loss, UDP** (LwM2M): datagrams are really dropped in either direction; CoAP and DTLS must retransmit.

- **Loss, TCP** (MQTT): dropping bytes would corrupt the stream, so a lost segment is delivered one retransmission timeout later instead (`-rto`, default 200 ms, doubling for consecutive losses), with order preserved – which is what the kernel's TCP would do.

Profiles: Excellent (0 ms, 0 %), Good (50 ms, 1 %), Poor (250 ms, 5 %), Very Poor (1000 ms, 10 %), or Custom. It is a teaching model: no jitter, bandwidth limit or reordering.

9.2 Kernel model (Linux/WSL)

`scripts/netem.sh` applies `tc netem` delay and loss to the server ports on the loopback interface (requires root). The dashboard's own network controls should then be set to 0 ms / 0 %.

10 Security experiments

	Security off	Security on
MQTT	TCP, port 1883	TLS 1.3 (or 1.2), port 8883, server certificate verified
LwM2M	CoAP/UDP, port 5683	DTLS 1.2 PSK, port 5684

The broker certificate is signed by a lab CA committed in `servers/mosquitto/certs` (lab use only; `gen-certs.sh` regenerates it). For DTLS the PSK must also be known to Leshan; the dashboard registers it through Leshan's REST API before a secure run and removes it before a plain run, because Leshan rejects a plain-CoAP registration with **4.03 Forbidden** for an endpoint that has a PSK. Unsupported modes (client certificates, RPK, OSCORE, bootstrap) are not offered in the UI.

11 Simulated OTA

Clearly labelled as a **simulation**: it does not model STM32 flash, banks, bootloaders, signing or rollback.

```
Server  -- GET /ota/manifest?image=X --> {version, size, sha256}  (firmware available)
Device  -- GET /ota/X  Range: bytes=a-b --> one chunk per request  (download)
Device  -- write chunk --> results/ota_storage/slot_b.bin          (storage)
Device  -- SHA-256(slot B) == manifest.sha256 ?                    (verify)
Device  -- write version --> active_version.txt                    (activate)
```

The download uses the same link and network simulation as the protocols, so latency and loss slow it down in the same way. Chunked range requests (default 1 KiB) mimic a modem-attached MCU fetching an image block by block; a failed chunk is retried up to three times. Interrupt aborts the download (the running version is unchanged); reset wipes slot B and returns to v1.0.0. Reported: firmware size, download, verify and total time, wire bytes, chunks, retries, completion or failure reason. Images are deterministic pseudo-random data with a small header; besides `firmware_v1/v2.bin` the lab generates `fw_<N>k.bin` on demand for size sweeps.

12 Verification approach

- **Unit tests** (`ctest`, 5 suites): sensor range, smoothness and reproducibility; nearest-rank percentiles and "not measured" serialisation; exact payload padding; the network model (per-packet delay, UDP loss rate, TCP never drops); CoAP decoding, LwM2M naming, retransmission detection and latency from first send.

- **Real-server tests**: every MQTT QoS level, TLS 1.3 with verification, every LwM2M operation (Register, Read, Write, Execute, Observe, Cancel, Notify, Update, Deregister), DTLS-PSK, OTA with and without impairment and with interrupt – against Mosquitto 2.1.2 and Leshan 2.0.0-M18.
- **Physics checks of the network model**: at 200 ms one-way latency QoS 1 measured $\approx$ 415 ms (one round trip), QoS 2 $\approx$ 825 ms (two), TLS 1.3 handshake $\approx 2L$, DTLS 1.2 PSK handshake $\approx 6L$.
- **Cross-checks**: the device's own counts compared with the server-side view (lab MQTT subscriber, Leshan's event stream), e.g. 10 notifications ACKed on the device and 10 NOTIFICATION events from Leshan.
- **Browser tests**: all tabs exercised in Chrome, including the experiment runner.
- **Linux build**: clean build and passing tests under WSL (GCC 15), plus a runtime smoke test of the lab and devices.

13 Problems found and how they were solved

Problem	Resolution
Port 1883 refused on Windows	Port range reserved by Hyper-V/WinNAT. All ports made configurable; documented the `netsh` reservation and alternate ports.
Wakaama notifications are NON	Converted to CON with RFC 7252 retransmission in the transport glue; NON kept as an option.
LwM2M REGISTER got 4.03	Stale PSK in Leshan after a DTLS run; the lab now removes it for plain runs and the device explains the error.
In-flight notifications counted as failed at the end of a run	Device stops producing when the count is sent or in flight, then waits for each to be ACKed or exhausted.
LwM2M "time to registered" inflated by the main-loop tick	Registration is checked immediately after each received packet.
LwM2M latency missed the outbound delay	Timestamps moved to before `link_send`, consistent with MQTT.
First network model charged latency only on direction changes (back-to-back packets skipped the delay)	Replaced with per-packet delay lines in both directions; verified against round-trip counts.
Dashboard slowed under heavy event bursts and background-tab throttling	Coalesced redraws, bounded series, event-driven experiment runner.
Mosquitto 2.1 deprecations / inline comments	Configuration cleaned (no `per_listener_settings`, comments on their own lines).

14 Measured results (development PC)

Windows 11, loopback, Mosquitto 2.1.2, Leshan 2.0.0-M18, MQTT QoS 1, LwM2M CON notifications, natural payload, interval 0.2 s. These are measurements from one machine, not general claims; loopback numbers show protocol overhead, not radio behaviour.

	MQTT		LwM2M	
Run	median	P95	median	P95
Benchmark, 100 messages, no impairment	0.38 ms	0.71 ms	1.17 ms	2.74 ms
Loss 1 % (50 messages)	0.37 ms	0.53 ms	1.02 ms	1.80 ms
Loss 2 %	0.37 ms	0.54 ms	1.07 ms	1.79 ms
Loss 5 %	0.37 ms	208 ms	1.07 ms	2769 ms
Loss 10 %	0.43 ms	213 ms	0.96 ms	6181 ms

All runs above delivered 100 % of messages.

	MQTT		LwM2M	
Security	off	TLS 1.3	off	DTLS 1.2
Time to ready	0.76 ms	12.7 ms	2.6 ms	25.3 ms
Wire bytes per message	158.7	207.0	36.8	94.8
Handshake bytes	–	2812	–	601

Interpretation

- With no impairment both protocols are sub-millisecond to a few milliseconds on loopback; LwM2M's slightly higher latency includes the CoAP exchange through Wakaama and Leshan.
- LwM2M carries far fewer bytes per message here (one resource value, compact SenML/CBOR, 4-byte CoAP header) than MQTT carrying the full JSON document over TCP.
- Under loss both still deliver everything, but recover very differently: TCP's 200 ms retransmission timer bounds MQTT's tail around 200 ms, while CoAP's 2–3 s ACK timeout with exponential backoff pushes LwM2M's P95 to seconds.
- Security costs a handshake (TLS 1.3: 1 round trip, 2.8 KB with the certificate; DTLS 1.2 PSK: 3 round trips, 0.6 KB) and per-record overhead on every message.

15 Known limitations

- The built-in network model has no jitter, reordering or bandwidth limit; use `tc netem` for packet-accurate impairment.
- LwM2M notifications are confirmable by design choice; stock Wakaama behaviour is available with `-notify-non`.
- OTA is a simulation; LwM2M Object 5 firmware update is not implemented.
- No bootstrap server, queue mode, client certificates, RPK or OSCORE.
- Lab TLS certificates and the default DTLS key are for the lab only.
- Not exercised on the development PC: Docker Compose (Docker not installed) and `netem.sh` (requires root in WSL).

16 Porting to STM32 + EC200U

Only `pal.c` touches the operating system. The rest is portable C:

PC (pal.c)	STM32 + EC200U
pal_now_us, pal_sleep_ms	SysTick or a timer; HAL/RTOS delay
pal_sock_connect	AT+QIOPEN=1,<id>,"TCP"|"UDP",host,port → +QIOPEN: <id>,0
pal_sock_send	AT+QISEND=<id>,<len>, data, SEND OK
pal_sock_recv	wait for +QIURC: "recv",<id>, then AT+QIRD
pal_sock_close	AT+QICLOSE=<id>
entropy for mbedTLS	STM32 RNG peripheral

Unchanged: `mqtt_client.c`, `lwm2m_client.c`, `objects.c`, `sensor.c` (or a real driver), `telemetry.c`, `bench.c`, `link.c` with a bare-metal mbedTLS configuration. Lab-only parts (`netsim.c`, `events.c`, `control.c`) are compiled out or redirected to a debug UART. Points to plan: an interrupt/DMA-driven UART ring buffer with an AT parser that handles unsolicited result codes; whether TLS runs on the MCU (full visibility) or in the modem (`AT+QSSLOPEN`, less MCU memory); trimming buffers; real time for certificate checks; LwM2M queue mode for PSM/eDRX.

A Repository layout

```
CMakeLists.txt, cmake/deps.cmake      build + dependency pinning
device/common/                        pal, link, netsim, sensor, telemetry,
                                      events, control, cli
device/mqtt/                          main.c, mqtt_client.c
device/lwm2m/                         main.c, lwm2m_client.c, objects.c,
                                      coap_trace.c
benchmark/                            bench.c (records, percentiles, CSV)
ota/                                  ota_device.c, make_firmware.c, fw_image.h
dashboard/server/                     lab.c, http.c, monitor.c, proc.c
dashboard/www/                        index.html, style.css, app.js, charts.js
servers/mosquitto/                    configs, lab certificates, gen-certs.sh
servers/leshan/                       Dockerfile
scripts/                              get/start servers (ps1, sh), netem.sh
tests/                                unit tests (ctest)
docs/                                 Markdown guides and this document
docker-compose.yml, README.md
```

B Command-line reference

Common device options

```
--host H  --port N  --id NAME  --interval 10s|500ms  --count N  --payload BYTES
--seed N  --timeout 10s  --secure  --ca FILE  --tls-name NAME  --tls12
--psk-id ID  --psk HEX  --profile excellent|good|poor|very-poor|custom
--latency MS  --loss PERCENT  --rto MS  --lab HOST:PORT|off  --ctl-port N
--results DIR  --run ID
```

Device-specific options

```
mqtt_device   --qos 0|1|2  --no-probe  --keepalive S  --topic PREFIX
              --user U  --pass P
lwm2m_device  --lifetime S  --notify-non  --ack-timeout 2s
ota_device    --image NAME  --chunk BYTES  --action update|reset
lab           --http-port 8000  --events-port 7000  --mqtt-host H
              --mqtt-port 1883  --mqtts-port 8883  --leshan-host H
              --leshan-port 8080  --coap-port 5683  --coaps-port 5684
```

```
              --www DIR  --results DIR
```

C Leshan REST calls used

```
GET    /api/clients                                        registered clients
GET    /api/clients/<ep>/<path>?timeout=5                  READ
PUT    /api/clients/<ep>/<path>?timeout=5                  WRITE
       {"id":5750,"kind":"singleResource","value":"kitchen","type":"string"}
POST   /api/clients/<ep>/<path>?timeout=5                  EXECUTE
POST   /api/clients/<ep>/<path>/observe                    OBSERVE
DELETE /api/clients/<ep>/<path>/observe?active=true        cancel observation
GET    /api/event                                          event stream (SSE)
PUT    /api/security/clients/                              add PSK
       {"endpoint":"lwm2m-dev-1","tls":{"mode":"psk","details":
        {"identity":"lwm2m-dev-1","key":"000102030405060708090a0b0c0d0e0f"}}}
DELETE /api/security/clients/<ep>                          remove PSK
```